DDoS Detection in Kubernetes using Machine and Deep Learning

Asher Siddique
Department of Software Engineering
FAST NUCES
ashersiddique127@gmail.com
Karachi, Pakistan

Mahnoor Amjad
Department of Software Engineering
FAST NUCES
mahnooramjadwork@gmail.com
Karachi, Pakistan

Supervisor: Shoaib Raza, Department of Cyber Security, FAST NUCES, Karachi

***Abstract*—In the recent years, cloud technology has advanced across all disciplines and used worldwide. Micro-service architecture, infamous for deploying services as per the user demands has also spread notoriously. Cloud is known for deploying on-demand services and micro-services are known for being the most scalable as the user traffic/demand grows. This is why these both go hand-in-hand because as the traffic grows new services can be deployed on demand. As with all technology, the responsibility of making cloud a secure environment to protect the users' interests and data is a primary concern.**

The build-out of cloud native applications and micro-service architectures across various systems and tech in the industry has brought about an intensity in the need for a set of effective mechanisms to deal with DoS (Denial of Service) and DDoS (Distributed Denial of Service) attacks in cloud native environments.

***Index Terms*—DDoS detection, Kubernetes, Machine Learning, Deep Learning, Container Orchestration, Cloud Security**

I. Introduction

With the rising adoption of cloud native applications & microservice architectures, Kubernetes has gained sufficient notability as a container orchestration platform that deploys micro-services as per user/customer demand. However, it is prone to DoS and DDoS attacks made vulnerable by its scalability and flexibility issues just like any other online system is, but what makes it harder to detect is its distributed nature. This leads to severe service disruptions, loss of revenue, & compromised customer trust. Classical machine learning has been employed for anomaly detection in cloud and containerized environments. Deep learning has been tried in detecting DoS/DDoS attack patterns. Challenges, however, remain in terms of real-time detection, scalability, minimizing false positives, and adapting to evolving attack patterns. Our review spans over two key research areas; classical machine learning for general anomaly detection in microservices and containers, and deep learning approaches specifically targeting DoS/DDoS detection in Kubernetes environments. After having identified the gaps in research and practice in developed and deployed systems, we aim on setting up a local cluster in Kubernetes to prepare a dataset comprised of syscalls and resource metrics upon which selected ML and DL models will be trained.

II. Related Work / Literature Review

A. Machine Learning (ML) Models for DoS and DSoS Detection

Classical Machine Learning has been explored and deployed to study patterns and detect anomalous trends or behaviors in Kubernetes environments. Anemogiannis [1] proposed a hierarchal system for anomaly detection by establishing baseline metrics for each Kubernetes resource based on activity over time. ML is used to study any deviations that occur in the multilayered system through the baseline metrics that act as thresholds. The abnormalities are scored according to severity with the top most crucial anomaly being flagged for immediate action and the lowest one being logged for trend analysis. The hierarchal structure helps with the cascading issues, as a problem in one resource impacts others, allowing for a targeted and more efficient response to anomalies. Du et al. [2] describes a system that applies supervised learning to classify system behavior based on performance data. It takes labelled sample data to train models and deploys different algorithms (Support Vector Machines (SVM), Random Forests, Naive Bayes, and k-Nearest Neighbors (k-NN)) to classify the behaviors. It then injects faults like high CPU usage to derive real-time instances of anomalies to improve the accuracy of the model to detect varied typed of anomalies in live deployment. The system performs time series analysis across multiple containers running the same micro-service to identify the specific container causing the issues. Zou et al.[3] proposed an enhanced version of Isolation Forest, where weights are assigned to each resource metric (e.g., CPU, memory) based on their relevance to anomaly detection. This weighted feature selection improves accuracy by adjusting the random selection process. The system automatically adjusts its monitoring interval, reducing overhead while maintaining real-time anomaly detection.

B. Deep Learning (DL) Approaches for Detection

Deep Learning models have been deployed to capture data that helps identify Dos and DDoS attacks that a cloud native environment might suffer from. Tien et al. [4] combines

Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to detect anomalies in Kubernetes-managed Docker environments. CNNs capture spatial patterns in data, helpful for identifying unusual behaviors in resource metrics, while RNNs are adept at temporal patterns, tracking sequences of container activities over time. CNNs analyze structured data, like container resource usage, to spot unusual patterns. RNNs retain information from previous steps, allowing them to model temporal dependencies. The model is trained on historical data to identify patterns in normal operations, and it flags deviations as potential anomalies. Additionally, it leverages an ensemble of deep learning methods to boost detection accuracy, reaching an effectiveness of around 96Lin et al. [5] proposed a classified distributed learning model that classified applications based on their behavior patterns in system calls which helps tailor the anomaly detection to each application type and solving the issue of the diverse anomaly behaviors and limited training data. The CDL framework deploys an auto-encoder that reconstructs expected behavior patterns. The reconstruction error between the input and output layers is how attacks are detected; higher the error, the bigger the attack. Forsberg [6] focuses on identifying problematic behavior and tracing the root cause of it in distributed micro-service environments. Identification is carried out through the use of Self-Organizing Maps (SOM) models, an unsupervised network technique for dimensionality reduction and pattern recognition in complex datasets. SOM models use performance metrics to identify any deviations and the anomalies are then classified based on these metrics. Root-cause analysis is done by a dependency graph of the micro-services, combined with inter-service causal analysis. An anomaly alignment algorithm further enhances detection accuracy by adjusting for timing discrepancies, which helps in matching the precise moments of anomaly occurrences between dependent services.

C. Combination of ML & DL Models in Attack Detection

Some studies define approaches that make use of both ML and DL for anomaly or attack detection in a Kubernetes environment. Harlicaj [7] provides us with two approaches combined in effort. The first one uses statistical models to detect baseline anomalies. These statistical techniques help identify deviations based on historical data patterns, useful for detecting web-based attacks. Classical ML classifiers (e.g., k-Nearest Neighbors or Decision Trees) are deployed for detection providing initial, low level identification with minimal overhead. The second approach introduces a neural network model, specifically suited for detecting more complex, non-linear patterns in request traffic. This DL model, potentially a recurrent neural network (RNN) or convolutional neural network (CNN), enhances the ability to recognize attack signatures embedded within web requests. By training on labeled data, the DL model learns nuanced patterns, which helps in identifying multi-step attacks or subtle anomalies that might not be apparent with basic statistical methods.

D. Current Gaps

Across the research papers, several common gaps emerge in anomaly and DDoD detection within Kubernetes and containerized environments. Some studies focus on deep learning models but encounter challenges related to computational demands, making them less suitable for real-time detection in resource-constrained settings. For instance, LSTM and auto-encoder-based approaches can be effective for capturing complex patterns but may strain processing resources in large-scale clusters. On the other hand, certain papers rely heavily on traditional ML models, which can be efficient but may lack the sophistication needed to handle evolving, multi-step attacks or non-linear patterns in web traffic. While ML models like Isolation Forests and SVMs offer solid baseline performance, they may fall short when applied to complex, dynamic data in micro-service environments. Additionally, while some approaches provide a multi-level detection framework to better suit Kubernetes' layered structure, they often overlook temporal analysis and adaptive feature selection, which are essential for accurately detecting anomalies in rapidly changing systems. Finally, interpretability remains a limitation in several deep learning models, making it challenging for administrators to understand and trust automated decisions in security-critical applications.

E. Base Paper and Future Directions

The paper, "A Novel Approach to Feature Collection for Anomaly Detection in Kubernetes Environment", provides us with our basis for research and implementation. The paper presents a new method to collect features based upon which anomalies related to different attacks, resource contentions and/or performance bottlenecks are identified. These features are resource metrics such as, memory, CPU, network I/O etc, being utilized across different Kubernetes resources. Darwesh et al. [8] introduces an event-driven data collection technique that collects and compiles data per every change in configuration, providing a more detailed real-time view of the system as compared to periodic polling and helps in identifying issues where resource allocation and configurations could shift rapidly due to auto-scaling and other dynamic actions. The paper proposes that the system deploys multi-level analysis where anomalies are detected at every level i.e. container, node, pod. This hierarchal system proves effective for a Kubernetes architecture where anomaly detection can be localized as well as be carried out in a broader, systematic manner across the clusters. It further proposes the integration of machine learning models, including Isolation Forests and Long Short-Term Memory (LSTM) networks, for processing the collected features. Isolation Forests are particularly useful for their efficiency in high-dimensional, sparse data typical in microservice telemetry, while LSTMs handle sequential dependencies and time series data to identify patterns indicative of anomalous behavior. Darwesh et al. [8] prove their approach through the improvements in accuracy and a significant reduction in false positives, attributed to the optimized feature selection and the hierarchical anomaly

detection strategy. We will be building upon the real time monitoring that the paper explains which can be a significant step towards detecting anomalous behavior in any environment as well that of Kubernetes. The novelty in the feature collection proposed also helps greatly in identifying issues. By adapting the selected features to the environment's current state, the system minimizes data collection overhead, making it efficient for cloud-native applications that run on Kubernetes. The hierarchical anomaly detection structure aligns closely with the Kubernetes architecture, offering a unique perspective among conventional methods, which often do not account for Kubernetes' layered setup.

III. Methodology / Proposed System

A. FYP-I – Simulating DDoS Attacks in a Protected Kubernetes Environment

FYP-I involved deploying a Kubernetes cluster in a controlled local network. We handled the deployment process using kubeadm, and automated the setup with Ansible scripts to ensure consistency and reproducibility. The infrastructure consisted of one master node and three worker nodes, all running on bare-metal machines equipped with 12th Gen Intel Core i7 CPUs (16 cores) and 16GB DDR5 RAM. These specs were intentionally chosen to ensure stable performance under both normal and attack conditions.

For the CNI plugin, Calico was selected to manage network policies, while MetalLB provided external IPs for LoadBalancer services. HTTP traffic was routed through an Ingress controller, enabling more realistic application-layer traffic flow. As the target application, we deployed OWASP Juice Shop, an intentionally vulnerable e-commerce app that mimics a real-world microservice environment.

To simulate attacks, we created a separate Docker-based botnet system within the same network but isolated from the Kubernetes cluster. This botnet consisted of three attacker containers controlled by a Command and Control (CnC) server we configured to randomly trigger attack types and durations. Each attack session involved a structured window (ramp-up, peak, ramp-down) lasting 2–3 minutes, emulating real botnet behavior.

The attack vectors included:

- HTTP flood via a modified goldeneye.py script
- TCP, UDP, and ICMP floods using hping3

For telemetry, a multi-layered monitoring setup was implemented:

- Falco on each node to log system calls (accept, connect, sendto, etc.)
- Prometheus Node Exporter for resource metrics (CPU, disk I/O, network throughput)
- A custom Go-based Node Agent that I developed to aggregate and label data from both sources

The Node Agent assigned labels based on the current mode (e.g., normal_traffic, http_flood, etc.) and transmitted them to the CnC server, which collected and stored everything into a structured CSV with 50+ features per row, sampled every 3 seconds.

Despite building a fully functional attack and telemetry pipeline, the results were underwhelming. The system metrics (CPU, sockets, network) didn't vary significantly between normal and attack periods. It became clear that Kubernetes' native protections were absorbing most of the DDoS traffic, masking its effects before they could visibly impact the nodes or application.

Although FYP-I validated our infrastructure and data pipeline—including the custom Node Agent and attack automation—this limitation required a rethinking of the overall strategy. That realization was tackled in FYP-II.

B. FYP-II – Redesigning for Realistic DDoS Behavior and Feature-Rich Data

Having identified the masking effect of Kubernetes' built-in mitigations in FYP-I, we initiated a full architectural overhaul in FYP-II. This time, MetalLB was removed to eliminate load-balancing abstractions and designed a new custom HTTP server from scratch to simulate a CPU-bound workload. This server hashed incoming requests repeatedly, ensuring visible computational strain at the node level—a key improvement over Juice Shop, which barely registered performance degradation under attack. We also rewrote the attack logic. Instead of relying on generalized tools, we built Python-based HTTP clients that maintained long-lived connections and launched high-frequency requests to trigger socket exhaustion. For TCP and UDP floods, iperf3 was used to achieve controlled high-throughput traffic. ICMP was dropped from scope to focus entirely on HTTP, TCP, and UDP—attack vectors that directly stress the transport and application layers. To address the issue of low metric variation, data collection was restructured around alternating 4–8 minute time windows. Each window involved either normal or attack traffic, resulting in three dedicated datasets: normal + http, normal + tcp, and normal + udp. We designed an expanded schema with metadata columns such as attack_mode, attack_sublabel, attack_phase, start_time, and end_time to capture full context for each interval. We also upgraded the telemetry processing pipeline. Instead of relying on raw syscalls and Prometheus metrics, statistical derivatives were computed—mean, standard deviation, min/max, and deltas—for key syscalls like mmap, accept, connect, socket, and more. Prometheus data was likewise processed to reflect trends over time rather than point-in-time snapshots. To improve data quality, we conducted feature analysis and removed four features that showed zero variance across sessions: node_sockstat_TCP_inuse, node_sockstat_UDP_inuse, node_network_receive_errs_total, and node_netstat_Tcp_CurrEstab. This refinement noticeably improved model accuracy during testing. With this new dataset, we retrained the models. This time, the differences were clear:

- Syscall activity (especially accept, connect, recvfrom) spiked during attacks

- CPU and memory metrics responded accurately to HTTP hash-loads
- Socket-related metrics visibly escalated under TCP/UDP stress

The enhanced dataset provided a much more realistic and detailed picture of what a node under DDoS looks like. We evaluated both traditional ML models (Random Forests, Decision Trees) and deep learning models (LSTM, GRU). After feature refinement and 100 training epochs:

- Random Forest achieved 99% accuracy
- Decision Tree reached 95%
- LSTM and GRU achieved 95–96%, showing strong potential for sequence-based anomaly detection

Finally, thanks to the clean labeling and dataset segmentation we designed, the models had a reliable ground truth to train on. By isolating protocols and excluding idle periods, we were able to reduce noise significantly—ensuring that our detection systems focused on real anomalies rather than ambient traffic.

C. System Architecture

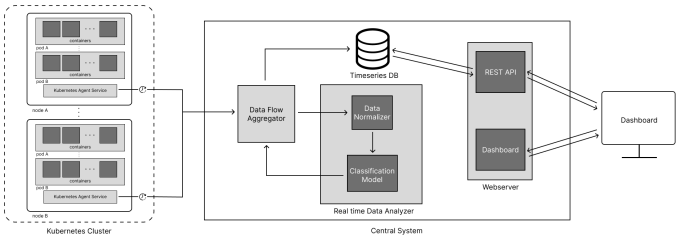

Fig. 1. System Design

D. Dashboard Snapshot

Fig. 2. Grafana Dashboard

IV. Results and Discussion

A. FYP-I Findings

After successfully deploying a four-node Kubernetes cluster and implementing a botnet simulation environment with a Command and Control (CnC) server, we conducted extensive model evaluations using the collected dataset. This dataset consisted of over 55,000 rows of telemetry data, captured at 3-second intervals, and included system calls and resource metrics from all worker nodes during both normal operations and various DDoS attack scenarios (HTTP, UDP, TCP, and ICMP floods). Preliminary data analysis revealed that although Kubernetes effectively mitigated much of the attack traffic, distinct patterns persisted in system call and resource utilization data. We implemented and evaluated four machine learning models using a 90-10 train-test split, validating their performance through 5-fold cross-validation. The Random Forest Classifier, configured with 100 estimators, delivered exceptional performance with 99.8% accuracy, precision, recall, and F1 score. It trained in approximately 1.01 seconds and tested in 0.04 seconds, making it highly efficient for real-time detection scenarios. Feature importance analysis highlighted that network-related system calls such as accept, connect, and sendto, alongside certain network metric features, were key to identifying attack signatures. The Decision Tree Classifier also performed strongly, achieving 95.4% accuracy, with similar precision, recall, and F1 scores, while requiring minimal computation time (0.07 seconds for training and 0.002 seconds for testing). In contrast, non-tree-based models struggled with the classification task. The Support Vector Machine (SVM) with a linear kernel achieved only 48.6% accuracy, and Logistic Regression performed even worse at 43.0%. Both models had poor performance across all evaluation metrics and longer training times, with SVM taking 3.32 seconds to train. Confusion matrix analysis reinforced these results, showing that while tree-based models effectively distinguished between attack types, SVM and Logistic Regression frequently misclassified them.

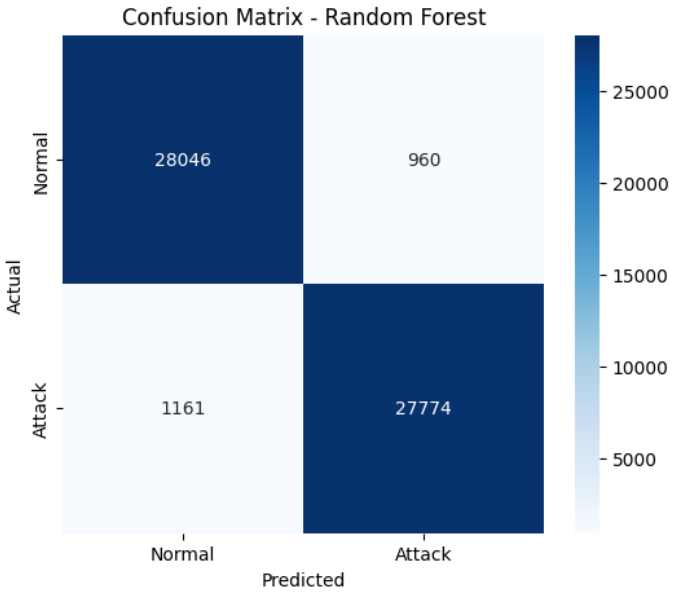

Fig. 3. Confusion Matrix for SVM

This significant performance gap indicated that the relationship between the telemetry metrics and attack classes was

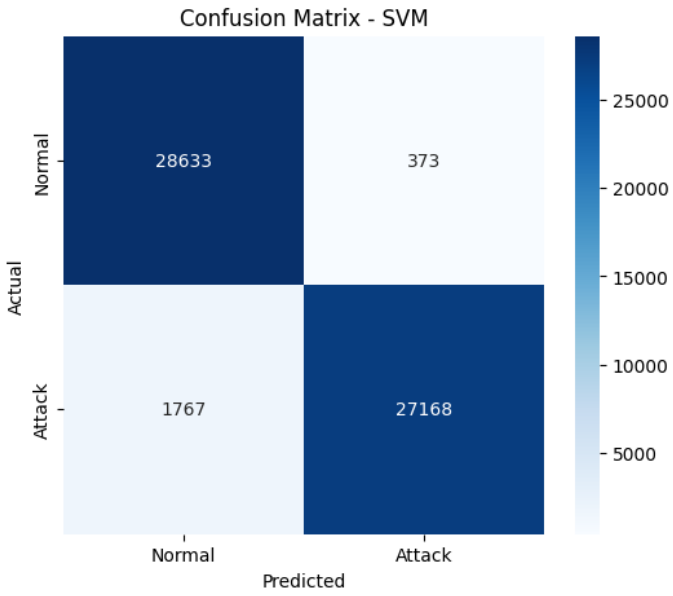

Fig. 4. Confusion Matrix for Random Forest

highly non-linear, favoring the hierarchical decision-making capabilities of tree-based models. However, the high accuracy of the Random Forest model raised concerns about potential overfitting, despite cross-validation being used. These insights informed our decision to explore more robust models in FYP-II. Nevertheless, FYP-I successfully demonstrated that even with Kubernetes' built-in mitigation absorbing much of the attack impact, detectable patterns remained in the telemetry data. These results validated our data collection approach and agent architecture while highlighting the effectiveness of tree-based models for detecting DDoS attacks in Kubernetes environments.

B. FYP-II Findings

In FYP-II, we redesigned the system architecture by removing MetalLB's load balancing abstraction and deploying a custom CPU-intensive HTTP server, which significantly improved the differentiation between normal and attack traffic. The refined telemetry pipeline generated a new dataset that served as the foundation for training and evaluating deep learning models, particularly designed to capture the temporal patterns of DDoS attacks in Kubernetes. We began with thorough preprocessing, as documented in the FYP_Final.ipynb notebook. Four low-variance features (node_sockstat_TCP_inuse, node_sockstat_UDP_inuse, node_network_receive_errs_total, and node_netstat_Tcp_CurrEstab) were removed, and the remaining features were normalized using StandardScaler. We restructured the data into time-series sequences suitable for Recurrent Neural Networks (RNNs), and encoded the target variable by mapping attack types to numerical values. To reduce noise, we segmented the dataset by attack type (HTTP, TCP, UDP), allowing each model to focus on a single protocol. We implemented two deep learning models: Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU). Both used similar architectures, consisting of a 64-unit recurrent layer, a 0.2 dropout layer for regularization, a dense ReLU layer, a second dropout layer, and a softmax output layer for classification. We trained the models using categorical crossentropy loss and the Adam optimizer, testing performance across 20, 50, and 100 epochs. The LSTM model performed consistently well, achieving:

- 92% accuracy at 20 epochs,
- 94% at 50 epochs,
- and 95% at 100 epochs.

After feature refinement, performance improved further to:

- 94%, 95%, and 96%, respectively.

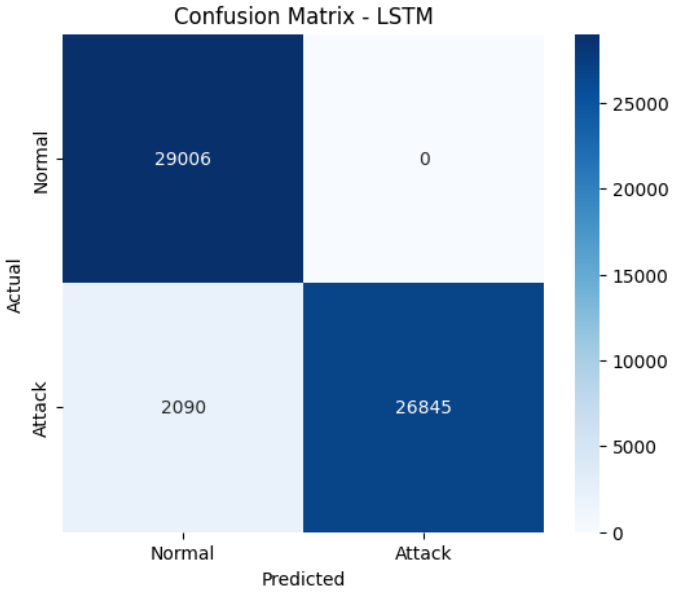

Fig. 5. Confusion Matrix for LSTM

The GRU model followed a similar trajectory:

- Starting with 91% accuracy at 20 epochs,
- Reaching 95% at 100 epochs.
- Post-feature refinement, it improved to 94% at 20 epochs and maintained 95% at 100 epochs.

Both models demonstrated smooth learning curves and consistent reductions in loss, indicating stable and reliable learning. Confusion matrix analysis confirmed that both architectures were highly effective in distinguishing between normal and attack traffic, with minimal misclassifications. Although the deep learning models did not surpass the Random Forest's 99% accuracy from FYP-I, they proved highly effective at capturing the temporal characteristics of DDoS behavior. While LSTM slightly outperformed GRU with a peak accuracy of 96%, the GRU's simpler architecture achieved nearly the same performance, suggesting that LSTM's additional complexity offered only marginal benefits for this task. This run's results validated our strategy of removing Kubernetes' protective abstractions and designing

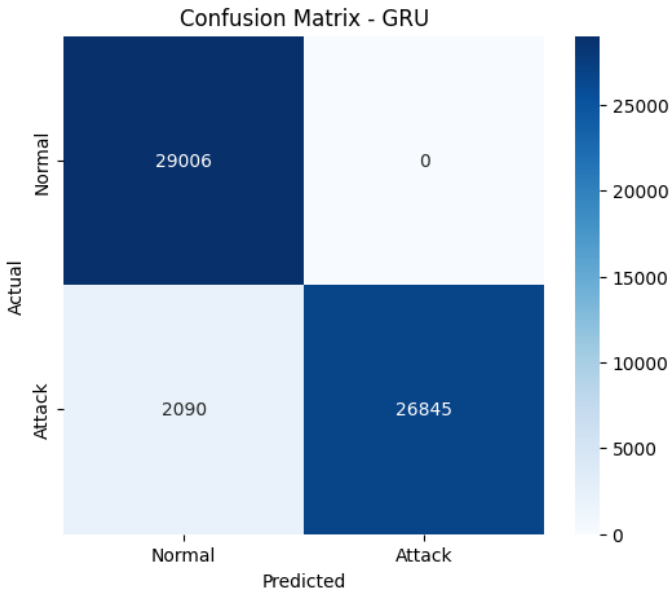

Fig. 6. Confusion Matrix for GRU

CPU-intensive workloads to generate clearer attack signatures. The deep learning models successfully detected these patterns in the enhanced dataset, showcasing the viability of recurrent architectures for real-time DDoS detection in Kubernetes. Taken together with FYP-I, our findings suggest that a hybrid detection system combining tree-based models for precision and deep learning models for temporal analysis would provide a robust solution for mitigating DDoS threats in modern containerized environments.

V. Conclusion

In this research, we dug into how machine learning (ML) and deep learning (DL) models can help detect DDoS attacks in a Kubernetes environment. In FYP-I, we saw just how powerful tree-based models can be, especially Random Forest, which nailed 99.8% accuracy. Even though Kubernetes' built-in protections did a good job absorbing some of the attack traffic, we still found clear attack patterns in the system calls and resource usage. On the flip side, models like Support Vector Machine (SVM) and Logistic Regression didn't perform as well, showing us just how non-linear the relationship is between telemetry data and attack types. FYP-II brought a fresh twist, where we took away Kubernetes' protective features and threw in a CPU-intensive server to generate clearer attack signals. This time, we turned to deep learning models—LSTM and GRU. While LSTM came out slightly ahead with 96% accuracy, GRU wasn't far behind with 95%. Both did a fantastic job of capturing the timing and sequence of DDoS behavior. When we put everything together, we found that a hybrid system—mixing tree-based models for accuracy and deep learning models for detecting attack patterns over time—would probably be the best approach for real-time DDoS detection in containerized environments like Kubernetes. These findings really show the potential of these models for improving security in cloud-native setups and provide a solid foundation for future research. Looking ahead, there's room for even more improvements. We could explore combining both ML and DL models into an ensemble system to get the best of both worlds. It would also be interesting to test these models with more diverse attack scenarios and larger datasets to see how well they scale. And of course, diving into more advanced Kubernetes mitigation techniques—like service meshes or smarter load balancing—could be the next step in improving detection accuracy even more.

References

[1] Anemogiannis, M. (2023). Hierarchical anomaly detection strategy for Kubernetes resources.
[2] Du, M., et al. (2018). DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning.
[3] Zou, H., et al. (2019). Docker container anomaly monitoring system based on optimized isolation forest algorithm.
[4] Tien, C. W., Liao, J. W., Chang, S. C., & Kuo, S. Y. (n.d.). KubAnomaly: Anomaly detection for the Docker orchestration platform with neural network approaches.
[5] Lin, Y., Tunde-Onadele, O., & Gu, X. (n.d.). CDL: Classified Distributed Learning for Detecting Security Attacks in Containerized App.
[6] Forsberg, J. (2022). Automatic anomaly detection and root cause analysis for microservice clusters.
[7] Harlicaj, E. (n.d.). Anomaly Detection of Web-Based Attacks in Microservices.
[8] Darwesh, G., Hammoud, J., & Vorobeva, A. A. (n.d.). A novel approach to feature collection for anomaly detection in Kubernetes environment and agent for metrics collection from Kubernetes nodes.